STAT 380: Ensemble Methods: Random Forest & Boosting

UAEU

Outlook of the unit

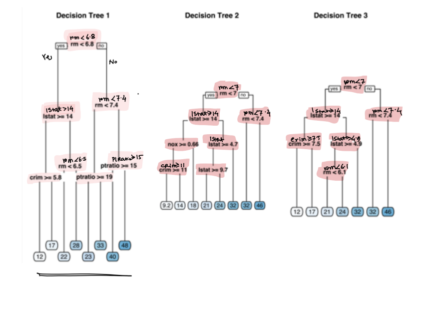
- **Bagging**
- **Random Forest**
- **Boosting**

Outline

Bagging

Standard Decision Tree may not be robust

- Three Entirely different Decision trees has been estimated based on three different sub samples of a Data set.



- Even if a couple of rows of the data are altered/ removed, the algorithm may find significantly different tree as an optimal solution.

Standard Decision Tree may not be robust

- The first few partitions may be similar in the top at each trees, they tend to differ significantly closer to the terminal nodes.

- The deeper nodes tend to over-fit the specific attributes of the sample data. Consequently, slight variation in the samples may lead to highly variable mechanism for estimating and predicting.

- Bootstrap Aggregating (Bagging) aims to address the instability in estimation and provide more robust solutions.

Bagging: Introduction

- The decision trees suffer from high variance.
- If we split the training data into two parts at random, and fit a decision tree to both halves, the results that we get could be quite different.
- A procedure with low variance will yield similar results if applied repeatedly to distinct data sets (like linear regression).
- Bootstrap aggregation, or bagging, is a general-purpose procedure for reducing the variance of a statistical learning method.
- Recall that given a set of n independent observations $Z_1, \ldots, Z_n$ each with variance σ^2, the variance of $\bar{Z}$ is given by σ^2/n.
- In other words, averaging a set of observations reduces variance.

- If $T_1, T_2, \ldots, T_B$ are uncorrelated with $E(T_i) = \mu$, and $\text{Var}(T_i) = \sigma^2 < \infty$, then

$$E(\overline{T}) = \mu, \quad \text{var}\left(\overline{T}\right) = \frac{\sigma^2}{B}$$

where $\overline{T} = \frac{1}{B}\sum_{i=1}^{B} T_i$

- The assumption that Z_i are uncorrelated is a crucial assumption uncorrelated.

Bagging = Bootstrap and aggregation

- A way to reduce the variance (increase prediction accuracy) of a method is to take many training sets, build prediction models using each training set, and average the predictions.
- This is not practical because we generally do not have access to multiple training sets.
- Instead, we can bootstrap, by taking repeated samples from the (single) training data set.

Bootstrap: An Example

Id	Price	Location	SqFt	Bedrooms	Bathrooms	Year Built
Id=1	350000	Downtown	1200	2	2	2010
Id=2	500000	Suburb	2000	4	3	2005
Id=3	250000	Rural	1500	3	2	1990
Id=4	600000	Downtown	1800	3	2	2015
Id=5	900000	Beachfront	2500	5	4	2020
Id=6	400000	Suburb	1400	3	2	1920

Bootstrap: An Example

Id	Price	Location	SqFt	Bedrooms	Bathrooms	Year Built
Id=1	350000	Downtown	1200	2	2	2010
Id=2	500000	Suburb	2000	4	3	2005
Id=3	250000	Rural	1500	3	2	1990
Id=4	600000	Downtown	1800	3	2	2015
Id=5	900000	Beachfront	2500	5	4	2020
Id=6	400000	Suburb	1400	3	2	1920

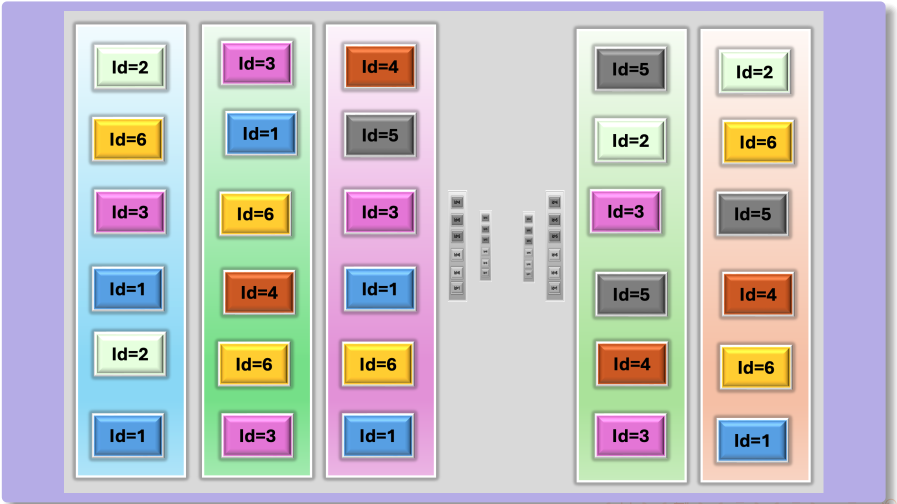
Bootstrap sample 1

Id=2	500000	Suburb	2000	4	3	2005
Id=6	400000	Suburb	1400	3	2	1920
Id=3	250000	Rural	1500	3	2	1990
Id=1	350000	Downtown	1200	2	2	2010
Id=6	400000	Suburb	1400	3	2	1920
Id=1	350000	Downtown	1200	2	2	2010

Bootstrap sample 2

Id=4	600000	Downtown	1800	3	2	2015
Id=5	900000	Beachfront	2500	5	4	2020
Id=4	600000	Downtown	1800	3	2	2015
Id=6	400000	Suburb	1400	3	2	1920
Id=5	900000	Beachfront	2500	5	4	2020
Id=3	250000	Rural	1500	3	2	1990

Bootstrap Samples: An Example



Bagging procedure for regression trees

1. Generate B different bootstrapped training data sets.
2. Train the method on the b-th bootstrapped training set in order to get $\hat{f}^{*,b}(x)$ for $b = 1, ..., B$ (for instance, construct regression trees without pruning).
3. Average all the predictions to obtain

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{f}^{*,b}(x).$$

Bagging in the context of classification trees

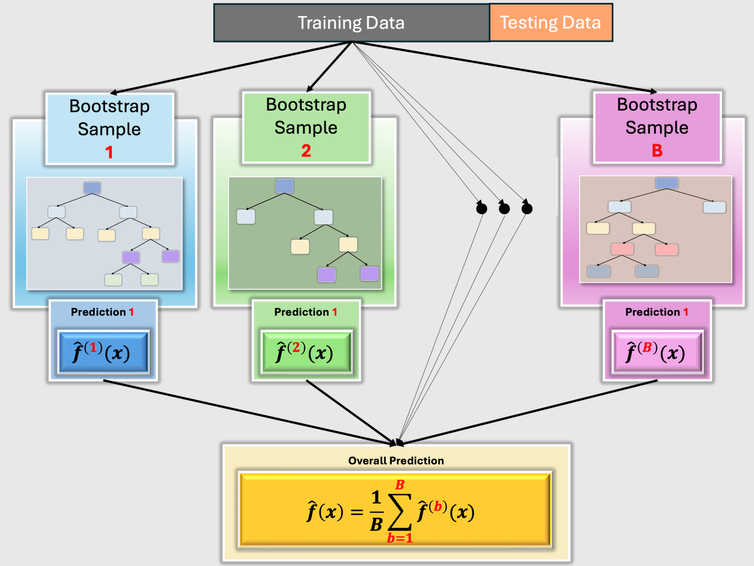
We have described the bagging procedure in the regression context, to predict a quantitative outcome Y.

Bagging for classification trees:

1. Generate B different bootstrapped training data sets.
2. Train the method on the b-th bootstrapped training set in order to record the class predicted by each of the B trees for a observation.
3. Take a majority vote: the overall prediction is the most commonly occurring majority class among the B predictions.

Comment:
Using a very large value of B will not lead to overfitting. In practice we use a value of B sufficiently large that the error has settled down, for instance $B = 500$.

Bagging: Bootstrap Aggregation



Variable importance measure

- When we bag a large number of trees, it is not possible to represent the resulting statistical learning procedure using a single tree.
- Thus, bagging improves prediction accuracy at the expense of interpretability.
- If there is a very strong predictor in a data set, then it is used to perform a first split in many trees grown from bagged samples.
- This leads to correlated bagged trees and averaging their predictions does not give substantial gains in accuracy.

Importance Measure of a Variable

- For a single tree, we can add up the decrements in Sum of Squared Errors associated with all the splits of a particular variable.

- For a Random Forest or Bagged trees we aggregate the sum of squared errors for all the trees in a Random Forest or Bagged Trees.

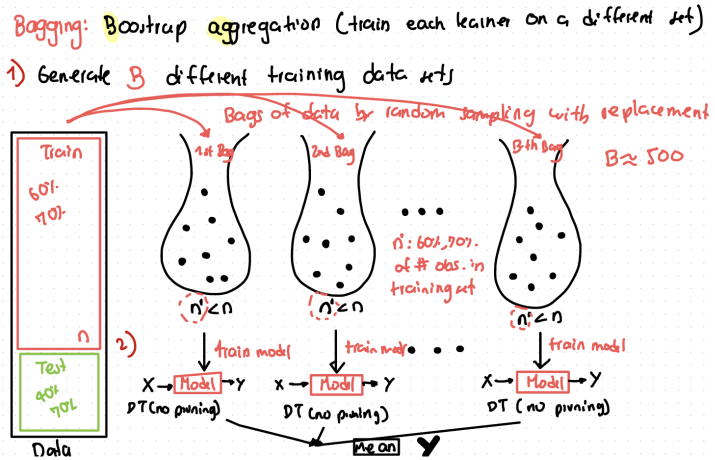
- There are R functions, “importance(·)”, and “varImpPlot(·)” in the “randomForest” R-package.

Relation Between a Variable and the Response

- A standard practice is to compute the average value of a prediction function for a fixed value of the target variable, while averaged over all the other variables.

- The above procedure is repeated over a range of values of the target variable, and is plotted to visualize the relation between a covariate and the response variable.

Bagging idea



Bagging regression trees in R: Boston housing data

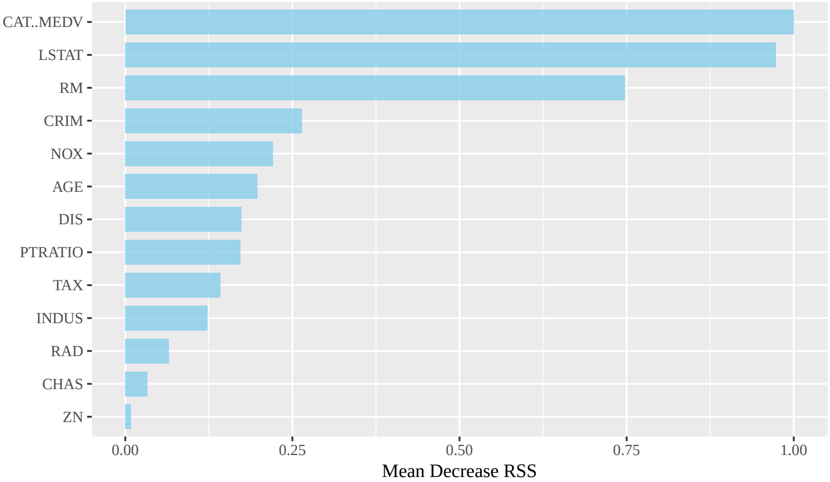
The `randomForest()`-command constructs a bagged tree and returns the results as an object.

```
# Loading libraries and the data
library(randomForest)

# Bagging a regression tree with all variables
> fit<-randomForest(MEDV~.,data=BostonH,mtry=13,
   importance =TRUE,subset=inTrain)
> fit
Call:
randomForest(formula = MEDV ~ ., data = BostonH, mtry =
   13, importance = TRUE, subset = inTrain)
   Type of random forest: regression
   Number of trees: 500
   No. of variables tried at each split: 13
```

- `mtry` indicates that all 13 predictors should be considered for each split of the tree.

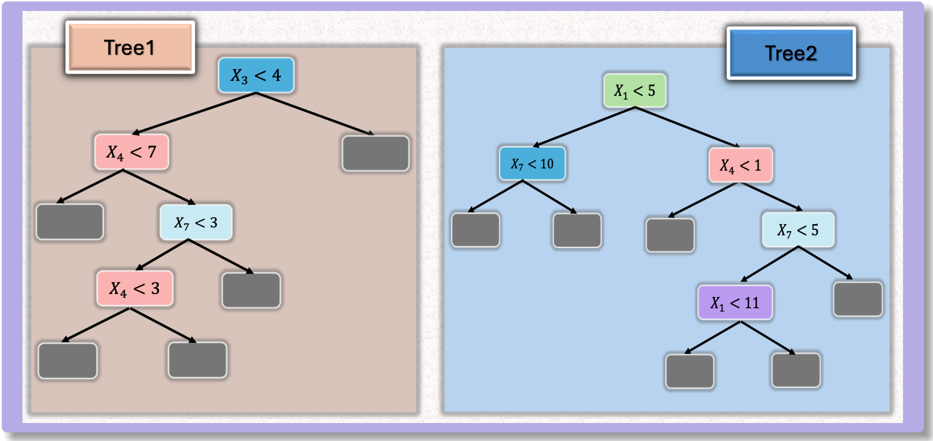
Bagging regression trees in R: Variable importance measure



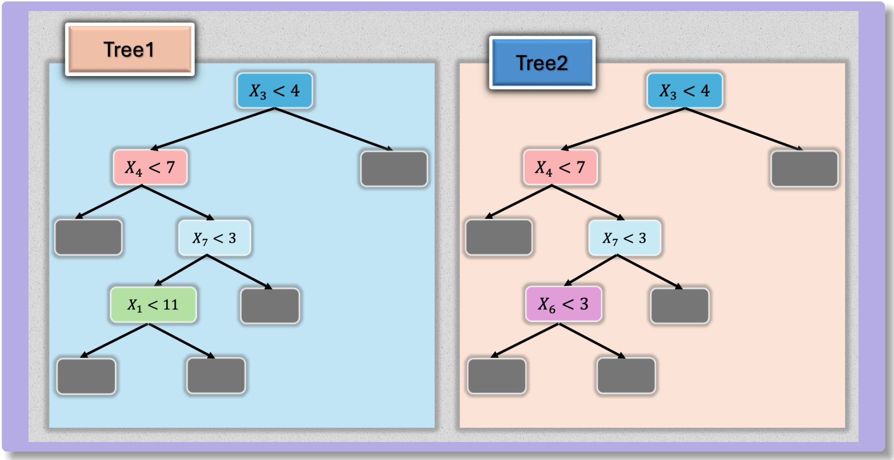
Outline

Random Forest

Correlated or Not?



Correlated or Not?



- If $T_1, T_2, \ldots, T_B$ are uncorrelated with $E(T_i) = \mu$, and $\text{Var}(T_i) = \sigma^2 < \infty$, then

$$E(\overline{T}) = \mu, \quad \text{var}\left(\overline{T}\right) = \frac{\sigma^2}{B}$$

where $\overline{T} = \frac{1}{B}\sum_{i=1}^{B} T_i$

- The assumption that T_i are uncorrelated is a crucial assumption uncorrelated.

How Correlation Affect the Variance of Sample Mean? A Special Case

- If $T_1, T_2, \ldots, T_B$ are correlated with $E(T_i) = \mu$, and $\text{Var}(T_i) = \sigma^2 < \infty$, $\text{cor}(T_i, T_j) = \rho > 0$, then

$$E(\overline{T}) = \mu, \quad \text{var}\left(\overline{T}\right) = \left(\frac{1-\rho}{B} + \rho\right)\sigma^2$$

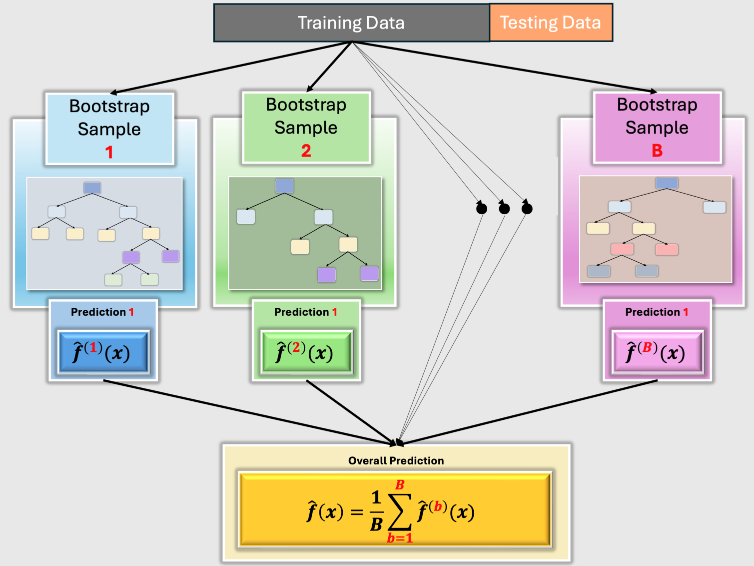
where $\overline{T} = \frac{1}{B}\sum_{i=1}^{B} T_i$

- If $\rho \approx 1, \rho < 1$, then $\text{var}\left(\overline{T}\right) \approx \sigma^2$
Average has almost same variability of a single TREE.
The Average does not lead to any significant improvement, and the predicted values are not very stable.

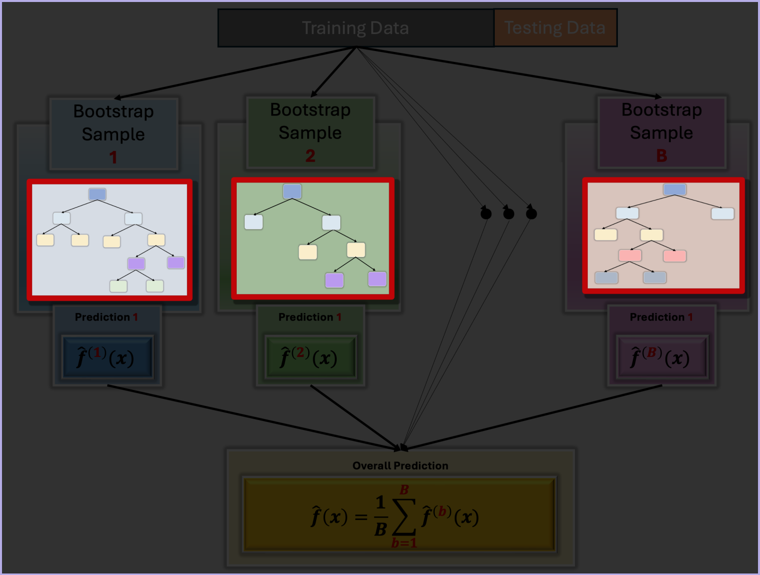
Bagging to random forests

- If there is a very strong predictor in a data set, then it is used to perform a first split in many trees grown from bagged samples.
- Consequently, all of the bagged trees will look quite similar to each other.
- This leads to correlated bagged trees and averaging their predictions does not give substantial gains in accuracy.
- Random forests provide an improvement over bagged trees
- It offers a solution to correlated bagged trees: allows a random sample of m out of p predictors at each split in a tree in order to expose different features to be explored by an algorithm.

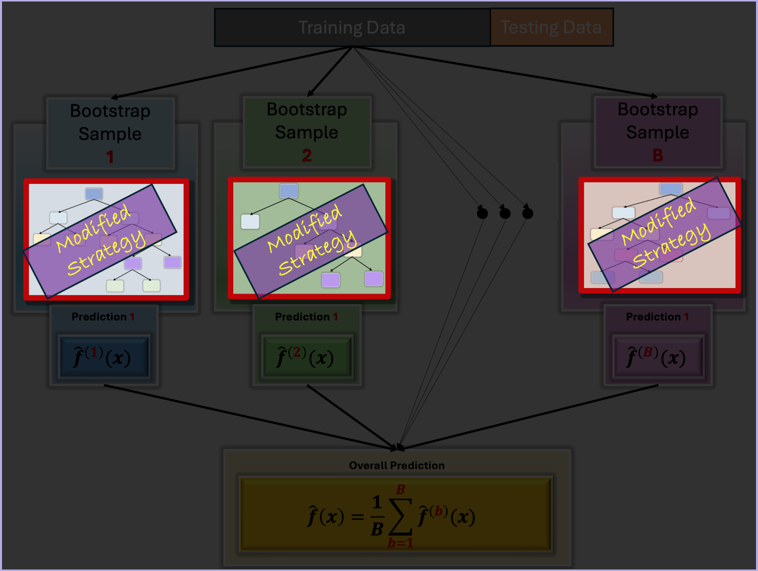
Random Forest



Random Forest



Random Forest



Random forests

- We build a number of decision trees on bootstrapped training samples.
- When building these decision trees, each time a split in a tree is considered, a random sample of m predictors is chosen as split candidates from the full set of p predictors.
- The split is allowed to use only one of those m predictors.
- A fresh selection of m predictors is taken at each split, and typically we choose $m \approx \sqrt{p}$ or by cross-validation.
- If a random forest is built using $m = p$, then this amounts simply to bagging.

Random Forest

	i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8
y	41	4	21	45	1	0	40	3
x1	86	60	53	93	67	59	66	76
x2	199	196	179	176	175	160	164	188
x3	27	42	49	29	53	52	24	56
x4	132	117	102	150	141	101	105	114
x5	506	537	528	540	521	505	504	523

$s \in \{ s_1 = x_{j,1} \quad s_2 = x_{j,2} \quad \cdots\cdots\cdots \quad s_n = x_{j,2} \}$

$j = 1, \ldots, 8$

	i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8
x1	1282	2560	2770	2019	2692	2767	2185	2261
x2	2235	2744	2734	2613	2769	2770	2341	2715
x3	2284	313	1392	1580	1965	1207	2770	2463
x4	2325	2679	2341	2019	2735	2770	2560	2765
x5	2769	2700	2695	2019	2491	2284	2770	2760

$$s \in \{ \; s_1 = x_{j,1} \quad s_2 = x_{j,2} \quad \cdots\cdots\cdots \quad s_n = x_{j,2} \; \}$$

$j = 1, \ldots, p$

$X_j < s$

$R_1(j, s_i)$

$X_j \geq s$

$R_2(j, s_i)$

$$C(j, s_i) := \sum_{i \in R_1(j, s_i)} \left(y_i - \widehat{y}_{R_1} \right)^2 + \sum_{i \in R_2(j, s_i)} \left(y_i - \widehat{y}_{R_2} \right)^2$$

Random Forest

$s \in \{\ s_1 = x_{j,1} \quad s_2 = x_{j,2} \quad \cdots\cdots\cdots \quad s_n = x_{j,2}\ \}$

$j = 1, \ldots, 8$

	i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8
x1	1282	2560	2770	2019	2692	2767	2185	2261
x2	2235	2744	2734	2613	2769	2770	2341	2715
x3	2284	313	1392	1580	1965	1207	2770	2463
x4	2325	2679	2341	2019	2735	2770	2560	2765
x5	2769	2700	2695	2019	2491	2284	2770	2760

The randomly selected variables to be considered are X_1, X_3.

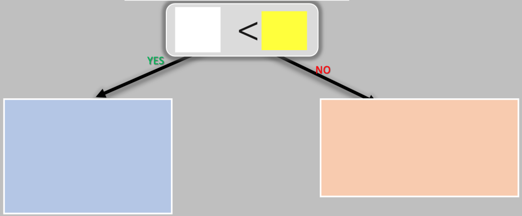
Random Forest

	i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8
y	41	4	21	45	1	0	40	3
x1	86	60	53	93	67	59	66	76
x2	199	196	179	176	175	160	164	188
x3	27	42	49	29	53	52	24	56
x4	132	117	102	150	141	101	105	114
x5	506	537	528	540	521	505	504	523

$s \in \{ s_1 = x_{j,1} \quad s_2 = x_{j,2} \quad \cdots\cdots \quad s_n = x_{j,2} \}$

$j = 1, \ldots, 8$

	i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8
x1	1282	2560	2770	2019	2692	2767	2185	2261
x2	2235	2744	2734	2613	2769	2770	2341	2715
x3	2284	313	1392	1580	1965	1207	2770	2463
x4	2325	2679	2341	2019	2735	2770	2560	2765
x5	2769	2700	2695	2019	2491	2284	2770	2760



Random Forest

	i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8
y	41	4	21	45	1	0	40	3
x1	86	60	53	93	67	59	66	76
x2	199	196	179	176	175	160	164	188
x3	27	42	49	29	53	52	24	56
x4	132	117	102	150	141	101	105	114
x5	506	537	528	540	521	505	504	523

$s \in \{ s_1 = x_{j,1} \quad s_2 = x_{j,2} \quad \cdots\cdots \quad s_n = x_{j,2} \}$

$j = 1, \ldots, 8$

	i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8
x1	1282	2560	2770	2019	2692	2767	2185	2261
x2	2235	2744	2734	2613	2769	2770	2341	2715
x3	2284	313	1392	1580	1965	1207	2770	2463
x4	2325	2679	2341	2019	2735	2770	2560	2765
x5	2769	2700	2695	2019	2491	2284	2770	2760

$$X_3 < 42$$

YES

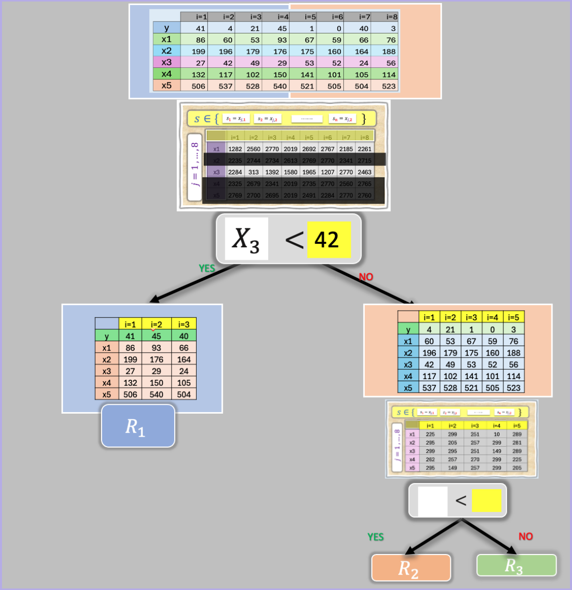
	i=1	i=2	i=3
y	41	45	40
x1	86	93	66
x2	199	176	164
x3	27	29	24
x4	132	150	105
x5	506	540	504

R_1

NO

	i=1	i=2	i=3	i=4	i=5
y	4	21	1	0	3
x1	60	53	67	59	76
x2	196	179	175	160	188
x3	42	49	53	52	56
x4	117	102	141	101	114
x5	537	528	521	505	523

Random Forest



	i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8
y	41	4	21	45	1	0	40	3
x1	86	60	53	93	67	59	66	76
x2	199	196	179	176	175	160	164	188
x3	27	42	49	29	53	52	24	56
x4	132	117	102	150	141	101	105	114
x5	506	537	528	540	521	505	504	523

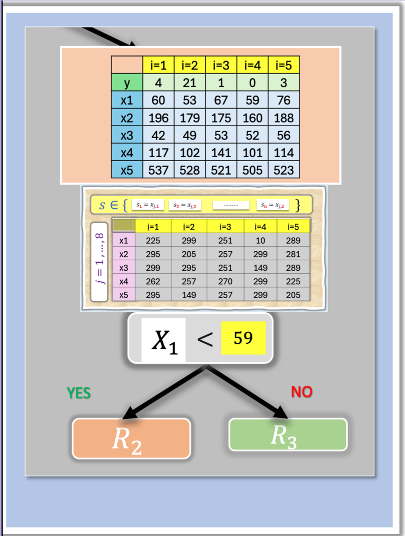
	i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8
x1	1282	2560	2770	2019	2692	2767	2185	2261
x2	2235	2744	2734	2613	2769	2770	2341	2715
x3	2284	313	1392	1580	1965	1207	2770	2463
x4	2325	2679	2341	2019	2735	2770	2560	2765
x5	2769	2700	2695	2019	2491	2284	2770	2760

	i=1	i=2	i=3
y	41	45	40
x1	86	93	66
x2	199	176	164
x3	27	29	24
x4	132	150	105
x5	506	540	504

	i=1	i=2	i=3	i=4	i=5
y	4	21	1	0	3
x1	60	53	67	59	76
x2	196	179	175	160	188
x3	42	49	53	52	56
x4	117	102	141	101	114
x5	537	528	521	505	523

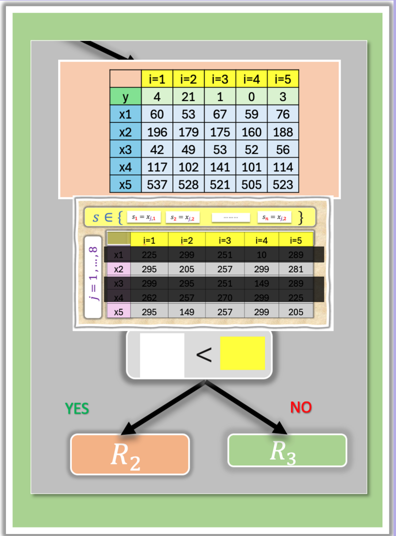
	i=1	i=2	i=3	i=4	i=5
x1	225	299	251	10	289
x2	295	205	257	299	281
x3	299	295	251	149	289
x4	262	257	270	299	225
x5	295	149	257	299	205

Standard Strategy & Random Forest



	i=1	i=2	i=3	i=4	i=5
y	4	21	1	0	3
x1	60	53	67	59	76
x2	196	179	175	160	188
x3	42	49	53	52	56
x4	117	102	141	101	114
x5	537	528	521	505	523

	i=1	i=2	i=3	i=4	i=5
x1	225	299	251	10	289
x2	295	205	257	299	281
x3	299	295	251	149	289
x4	262	257	270	299	225
x5	295	149	257	299	205

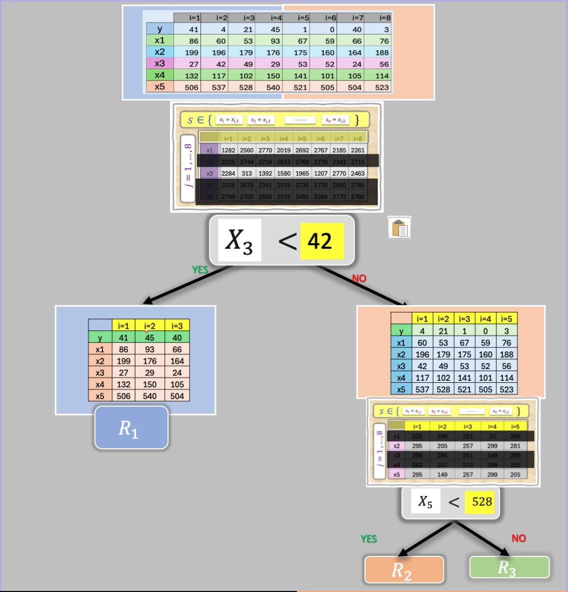


	i=1	i=2	i=3	i=4	i=5
y	4	21	1	0	3
x1	60	53	67	59	76
x2	196	179	175	160	188
x3	42	49	53	52	56
x4	117	102	141	101	114
x5	537	528	521	505	523

	i=1	i=2	i=3	i=4	i=5
[redacted]					
x2	295	205	257	299	281
[redacted]					
x5	295	149	257	299	205

The randomly selected variables to be considered are X_1, X_3.

Random Forest

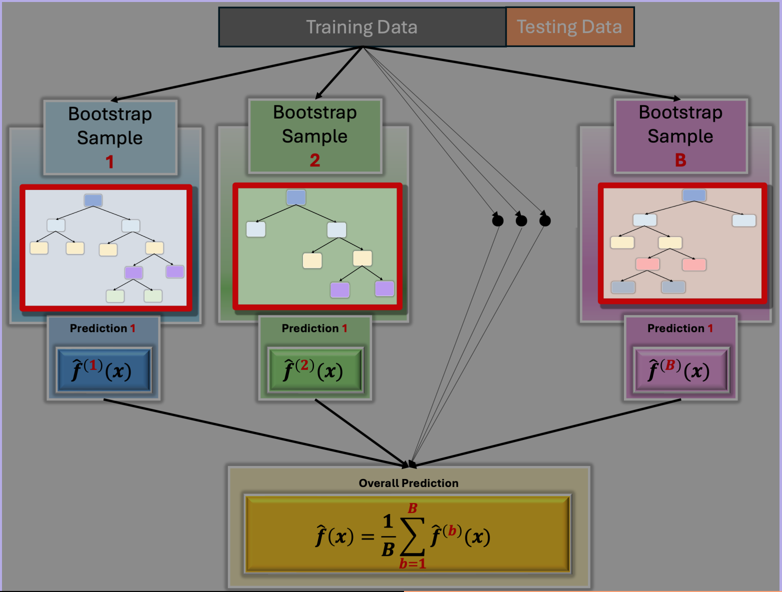


	i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8
y	41	4	21	45	1	0	40	3
x1	86	60	53	93	67	59	66	76
x2	199	196	179	176	175	160	164	188
x3	27	42	49	29	53	52	24	56
x4	132	117	102	150	141	101	105	114
x5	506	537	528	540	521	505	504	523

	i=1	i=2	i=3
y	41	45	40
x1	86	93	66
x2	199	176	164
x3	27	29	24
x4	132	150	105
x5	506	540	504

	i=1	i=2	i=3	i=4	i=5
y	4	21	1	0	3
x1	60	53	67	59	76
x2	196	179	175	160	188
x3	42	49	53	52	56
x4	117	102	141	101	114
x5	537	528	521	505	523

Random Forest



Random forests in R: Boston housing data

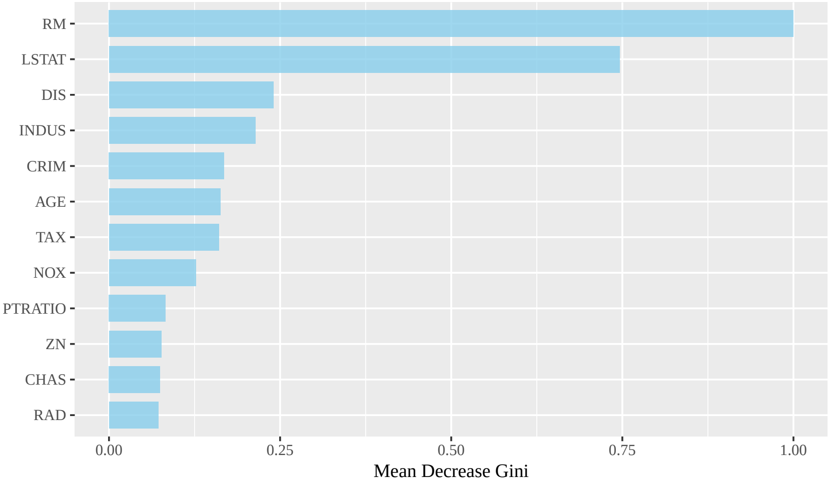
The `randomForest()`-command constructs random forests and returns the results as an object.

```
# Bagging a regression tree with all variables
> fit<-randomForest(MEDV_Fac~.,data=BostonH,mtry=sqrt(12)
   , importance =TRUE,subset=inTrain)
> fit

   Type of random forest: classification
   Number of trees: 500
   No. of variables tried at each split: 3
   OOB estimate of  error rate: 3.93%

Confusion matrix:
      Below Above class.error
Below   289     5   0.0170068
Above     9    53   0.1451613
```

Random forests in R: Variable importance measure



Pros and cons of random forest

Pros

- high predictive performance
- low chance of overfitting
- robustness
- easy to parallelize

Cons

- relatively long training time
- lack of interpretability (possible solution: variable importance measures)
- cannot incorporate missing values as easily as simple trees

Outline

Boosting

Boosting

Boosting is another technique for improving predictive accuracy of decision trees. It 'boosts' the performance of weak (only marginally better than random) classifiers.

Unlike bagging, in boosting trees are grown sequentially: a tree uses information from previously grown trees. Final classifier is built by combining sequentially updated classifiers.

Some popular boosting approaches:

- AdaBoost
- Stochastic Gradient Boosting

AdaBoost

Process:

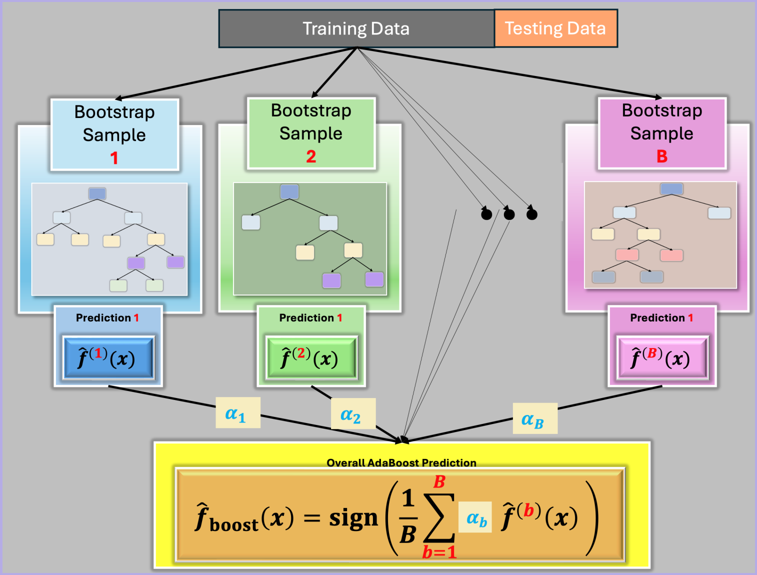
1. Initially each observation has the same starting weight $(1/n)$, an initial 'base' classifier is built and misclassification error is calculated.
2. Weights are updated in such a way that incorrectly predicted observations receive more weight and correctly classified less. A new classifier with updated weights is built and this step is repeated M times.
3. Final classifier is calculated as a weighted sum of M base classifiers.

Tuning parameters for boosting

- **Number of trees** should not be too large, because unlike bagging and random forest, boosting can overfit.
- **Shrinkage parameter, or learning rate** controls speed of learning. Typical values are 0.1 or 0.01.
- **Interaction depth** is the number of splits in each tree. One split per tree often works well.
- **Subsampling** uses only part of the data (typically $\frac{1}{2}$ or less) to improve predictive performance.

Use cross-validation to find promising values.

Adaptive Boosting (Binary Classification): Idea



Adaptive Boosting (Binary Classification) Algorithm

AdaBoost (Freund & Schapire, 1996)

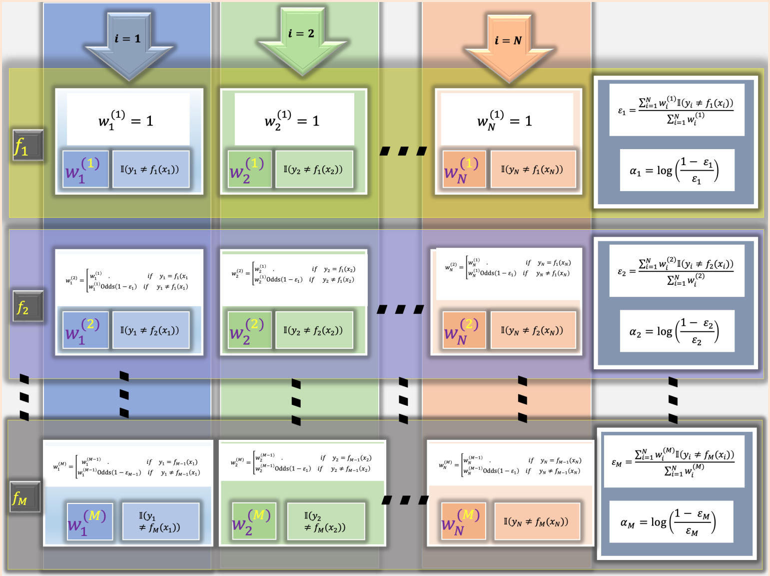
For this slide assume we have binary y. $y \in \{+1, -1\}$. $f(x_i) \in \{+1, -1\}$.

1. Initialize the observation weights $w_i = \frac{1}{N}$, $i = 1, 2, \ldots, N$.
2. For $m = 1$ to M repeat steps (a)–(d):
   - (a) Fit a classifier $f_m(x)$ to the training data using weights w_i.
   - (b) Compute weighted error of newest tree

$$\varepsilon_m = \frac{\sum_{i=1}^{N} w_i I(y_i \neq f_m(x_i))}{\sum_{i=1}^{N} w_i}.$$

   - (c) Compute $\alpha_m = \log\left[\frac{1-\varepsilon_m}{\varepsilon_m}\right]$ = $\text{Logit}(1-\varepsilon_m) = \text{Logit}(\text{Accuracy}_m)$
   - (d) Update weights for $i = 1, \ldots, N$:
     $w_i \leftarrow w_i \cdot \exp[\alpha_m \cdot I(y_i \neq f_m(x_i))]$
     and renormalize to w_i to sum to 1.
3. Output $f(x) = \text{sign}\left[\sum_{m=1}^{M} \alpha_m f_m(x)\right]$.

Adaptive Boosting (Binary Classification) Algorithm

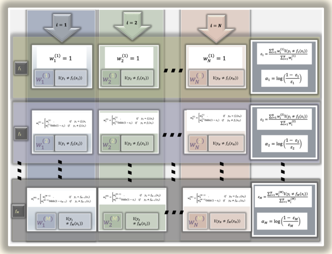


Adaptive Boosting (Binary Classification) Algorithm

f_1

f_2

f_M



$$\alpha_1 = \log\left(\frac{1-\varepsilon_1}{\varepsilon_1}\right)$$

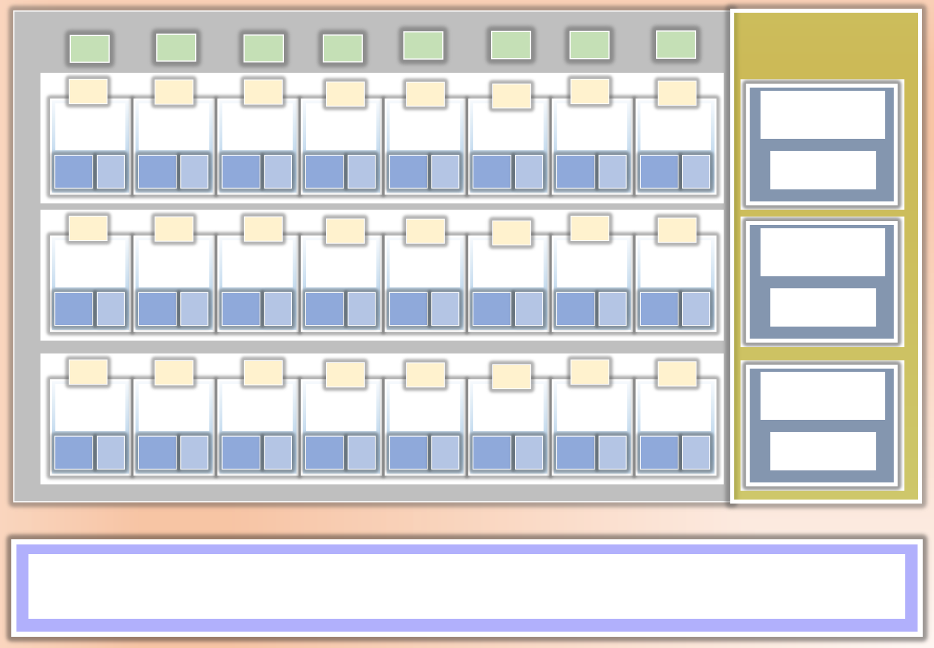
$$\alpha_2 = \log\left(\frac{1-\varepsilon_2}{\varepsilon_2}\right)$$

$$\alpha_M = \log\left(\frac{1-\varepsilon_M}{\varepsilon_M}\right)$$

$$\hat{f}_{\text{boost}}(x) = \text{sign}\left(\sum_{m=1}^{M} \alpha_m f_m(x)\right)$$

AdaBoost: A Toy Example

1	-1	1	1	-1	1	1	-1
1	1	1	1	1	1	1	1
-1	-1	1	1	1	1	1	1
1	-1	1	1	-1	1	-1	-1



Summary

- Decision trees are simple and interpretable models for regression and classification.
- However they are often not competitive with other methods in terms of prediction accuracy.
- Bagging, random forests (and boosting) are good methods for improving the prediction accuracy of trees.
- They work by growing many trees on the training data and then combining the predictions of the resulting ensemble of trees.
- Random forests and boosting are among the state-of-the-art methods for supervised learning - however their results can be difficult to interpret.

Thank You